第 4 章 动态规划

动态规划可以说是整个算法基础篇中，最难的一章：

- 首先，入门难。刚开始接触动态规划，你会觉得这玩意有点晦涩 + 玄学，稀里糊涂的就把问题解决了。不用担心，等做过五六道题目之后，就能理解动态规划是如何解决问题的；--- 坚持学、重复学
- 其次，题型多。不仅算法基础篇会讲到动态规划，在算法提高篇还会再讲。动态规划细分的话，可以分出十几种类型。所以，学习的成本是比较高的；
- 最后，题目难。在竞赛中，如果遇到动态规划的问题，只要不是经典题型，那么大概率就是以压轴题的形式出现。

但是，即使很难，我们也要慢慢去学习。想取得好成绩，动态规划是避不开的。不过大家放心，往后讲解的时候，我会循序渐进的进行，相信大家都是可以听得懂的。

1. 入门：从记忆化搜索到动态规划

1. 记忆化搜索

在搜索的过程中，如果搜索树中有很多重复的结点，此时可以通过一个 "备忘录"，记录第一次搜索到的结果。当下一次搜索到这个结点时，直接在 "备忘录" 里面找结果。其中，搜索树中的一个一个结点，也称为一个一个状态。

比如经典的斐波那契数列问题：

```
1  int f[N]; // 备忘录
2
3  int fib(int n)
4  {
5      // 搜索之前先往备忘录里面瞅瞅
6      if(f[n] != -1) return f[n];
7      if(n == 0 || n == 1) return f[n] = n;
8
9      // 返回之前，把结果记录在备忘录中
10     f[n] = fib(n - 1) + fib(n - 2);
11     return f[n];
12 }
```

2. 递归改递推

在用记忆化搜索解决斐波那契问题时，如果关注 "备忘录" 的填写过程，会发现它是从左往右依次填写的。当 i 位置前面的格子填写完毕之后，就可以根据格子里面的值计算出 i 位置的值。所以，整个递归过程，我们也可以改写成循环的形式，也就是递推：

```
int f[N]; // f[i] 表示：第 i 个斐波那契数

int fib(int n)
{
    // 初始化前两个格子
    f[0] = 0; f[1] = 1;

    // 按照递推公式计算后面的值
    for(int i = 2; i <= n; i++)
    {
        f[i] = f[i - 1] + f[i - 2];
    }

    // 返回结果
    return f[n];
}
```

3. 动态规划

动态规划（Dynamic Programming，简称DP）是一种用于解决多阶段决策问题的算法思想。它通过将复杂问题分解为更小的子问题，并存储子问题的解（通常称为“状态”），从而避免重复计算，提高效率。因此，动态规划里，蕴含着分治与剪枝思想。

上述通过**记忆化搜索**以及**递推**解决斐波那契数列的方式，其实都是动态规划。

> 注意：
>
> - 动态规划中的相关概念其实远不止如此，还会有：重叠子问题、最优子结构、无后效性、有向无环图等等。
> - 这些概念没有一段时间的沉淀是不可能完全理解的。可以等学过一段时间之后，再去接触这些概念。不过，这些概念即使不懂，也不影响做题~

在递推形式的动态规划中，常用下面的专有名词来表述：

1. 状态表示：指 `f` 数组中，每一个格子代表的含义。其中，这个数组也会称为 `dp` 数组，或者 `dp` 表。
2. 状态转移方程：指 `f` 数组中，每一个格子是如何用其余的格子推导出来的。

3. 初始化：在填表之前，根据题目中的默认条件或者问题的默认初始状态，将 `f` 数组中若干格子先填上值。

其实递推形式的动态规划中的各种表述，是可以对应到递归形式的：

- 状态表示 <---> 递归函数的意义；
- 状态转移方程 <---> 递归函数的主函数体；
- 初始化 <---> 递归函数的递归出口。

4. **如何利用动态规划解决问题**

第一种方式当然就是记忆化搜索了：

- 先用递归的思想去解决问题；
- 如果有重复子问题，就改成记忆化搜索的形式。

第二种方式，直接使用递推形式的动态规划解决：

1. 定义状态表示：

   一般情况下根据经验+递归函数的意义，赋予 `dp` 数组相应的含义。（其实还可以去蒙一个，如果蒙的状态表示能解决问题，说明蒙对了。如果蒙错了，再换一个试~）

2. 推导状态转移方程：

   根据状态表示以及题意，在 `dp` 表中分析，当前格子如何通过其余格子推导出来。

3. 初始化：

   根据题意，先将显而易见的以及边界情况下的位置填上值。

4. 确定填表顺序：

   根据状态转移方程，确定按照什么顺序来填表。

5. 确定最终结果：

   根据题意，在表中找出最终结果。

1.1 下楼梯

> **题目来源：** 洛谷
>
> **题目链接：** P10250 [GESP样题 六级] 下楼梯
>
> **难度系数：** ★

【题目描述】

> 顽皮的小明发现，下楼梯时每步可以走 1 个台阶、 2 个台阶或 3 个台阶。现在一共有 N 个台阶，你能帮小明算算有多少种方案吗？

【输入描述】

> 输入一行，包含一个整数 N。
>
> 对全部的测试点，保证 $1 \leq N \leq 60$。

【输出描述】

> 输出一行一个整数表示答案。

【示例一】

> 输入：
>
> 4
>
> 输出：
>
> 7

【示例二】

> 输入：
>
> 10
>
> 输出：
>
> 274

【解法】

因为上楼和下楼是一个可逆的过程，因此我们可以把下楼问题转化成上到第 n 个台阶，一共有多少种方案。

解法：动态规划

1. 状态表示：

   $dp[i]$ 表示：走到第 i 个台阶的总方案数。

   那最终结果就是在 $dp[n]$ 处取到。

2. 状态转移方程：

   根据最后一步划分问题，走到第 i 个台阶的方式有三种：

   a. 从 $i-1$ 台阶向上走 1 个台阶，此时走到 i 台阶的方案就是 $dp[i-1]$；

   b. 从 $i-2$ 台阶向上走 2 个台阶，此时走到 i 台阶的方案就是 $dp[i-2]$；

   c. 从 $i-3$ 台阶向上走 3 个台阶，此时走到 i 台阶的方案就是 $dp[i-3]$；

   综上所述，$dp[i]=dp[i-1]+dp[i-2]+dp[i-3]$。

3. 初始化：

   填 i 位置的值时，至少需要前三个位置的值，因此需要初始化 $dp[0]=1,dp[1]=1,dp[2]=2$，然后从 $i=3$ 开始填。

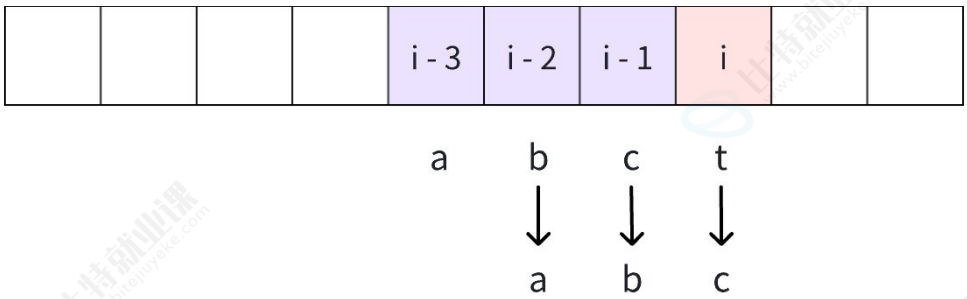
   或者初始化 $dp[1]=1,dp[2]=2,dp[3]=4$，然后从 $i=4$ 开始填。

4. 填表顺序：

   明显是从左往右。

动态规划的空间优化：

我们发现，在填写 $dp[i]$ 的值时，我们仅仅需要前三个格子的值，第 $i-4$ 个及其之前的格子的值已经毫无用处了。因此，可以用三个变量记录 i 位置之前三个格子的值，然后在填完 i 位置的值之后，滚动向后更新。



【参考代码】

```cpp
#include <iostream>

using namespace std;

```

```cpp
typedef long long LL;

const int N = 65;

int n;
// LL f[N]; // f[i] 表示：有 i 个台阶的时候，一共有多少种方案

int main()
{
    cin >> n;

    // 版本一：用数组的形式
    // 初始化
    // f[0] = 1; f[1] = 1; f[2] = 2;

    // for(int i = 3; i <= n; i++)
    // {
    //     f[i] = f[i - 1] + f[i - 2] + f[i - 3];
    // }

    // cout << f[n] << endl;

    // 版本二：空间优化
    LL a = 1, b = 1, c = 2;

    for(int i = 3; i <= n; i++)
    {
        LL t = a + b + c;
        a = b;
        b = c;
        c = t;
    }

    // 小细节
    if(n == 1) cout << b << endl;
    else cout << c << endl;

    return 0;
}
```

1.2 数字三角形

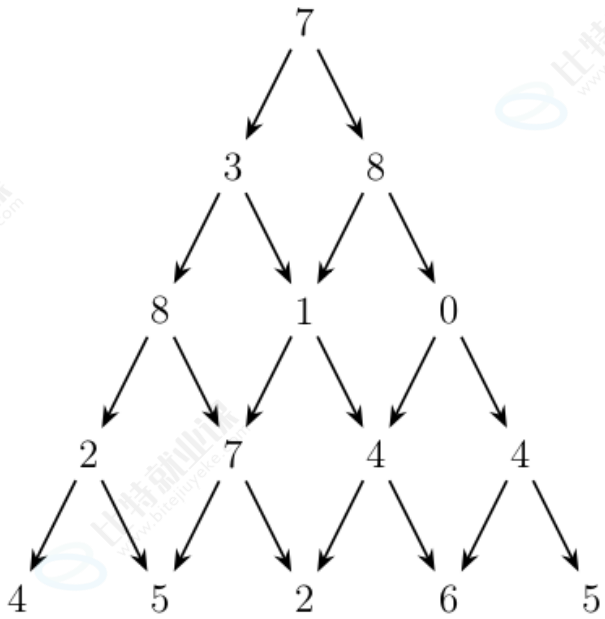
> **题目来源：** 洛谷
>
> **题目链接：** P1216 [USACO1.5] [IOI1994]数字三角形 Number Triangles

难度系数： ★

【题目描述】

观察下面的数字金字塔。

写一个程序来查找从最高点到底部任意处结束的路径，使路径经过数字的和最大。每一步可以走到左下方的点也可以到达右下方的点。



洛谷

在上面的样例中，从 $7 \to 3 \to 8 \to 7 \to 5$ 的路径产生了最大权值。

【输入描述】

第一个行一个正整数 r ,表示行的数目。

后面每行为这个数字金字塔特定行包含的整数。

对于 100% 的数据， $1 \leq r \leq 1000$ ，所有输入在 $[0, 100]$ 范围内。

【输出描述】

单独的一行,包含那个可能得到的最大的和。

【示例一】

输入：

```
5
7
3 8
8 1 0
```

> 2 7 4 4
>
> 4 5 2 6 5
>
> 输出：
>
> 30

【解法】

学习动态规划最经典的入门题。

解法：动态规划

1. 状态表示：

   $dp[i][j]$ 表示：走到 $[i,j]$ 位置的最大权值。

   那最终结果就是在 dp 表的第 n 行中，所有元素的最大值。

2. 状态转移方程：

   根据最后一步划分问题，走到 $[i,j]$ 位置的方式有两种：

   a. 从 $[i-1,j]$ 位置向下走一格，此时走到 $[i,j]$ 位置的最大权值就是 $dp[i-1][j]$；

   b. 从 $[i-1,j-1]$ 位置向右下走一格，此时走到 $[i,j]$ 位置的最大权值就是 $dp[i-1][j-1]$；

   综上所述，应该是两种情况的最大值再加上 $[i,j]$ 位置的权值：
   $dp[i][j]=max(dp[i-1][j],dp[i-1][j-1])+a[i][j]$。

3. 初始化：

   因为 dp 表被 0 包围着，并不影响我们的最终结果，因此可以直接填表。

   思考，如果权值出现负数的话，需不需要初始化？

   - 此时可以全都初始化为 $-\infty$，负无穷大在取 max 之后，并不影响最终结果。

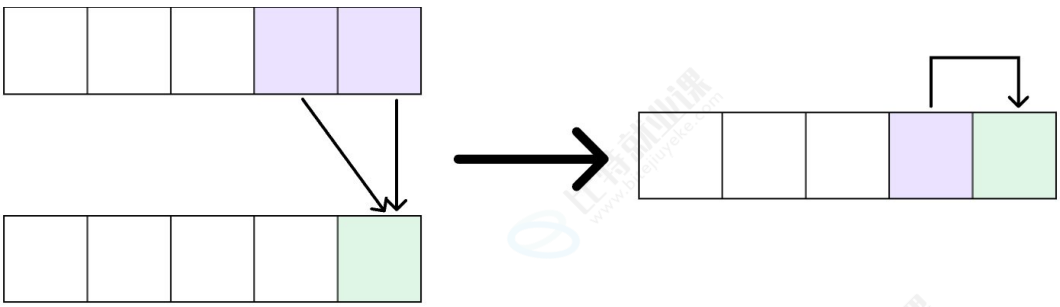
4. 填表顺序：

   从左往右填写每一行，每一行从左往右。

动态规划的空间优化：

我们发现，在填写第 i 行的值时，我们仅仅需要前一行的值，并不需要第 $i-2$ 以及之前行的值。

因此，我们可以只用一个一维数组来记录上一行的结果，然后在这个数组上更新当前行的值。



需要注意，当用因为我们当前这个位置的值需要左上角位置的值，因此滚动数组优化的时候，要改变第二维的遍历顺序。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 1010;

int n;
int a[N][N];
int f[N]; // f[i][j]: 从 [1, 1] 走到 [i, j] 时，所有方案下的最大权值

int main()
{
    cin >> n;

    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= i; j++)
            cin >> a[i][j];

    // 二维数组的版本
    // for(int i = 1; i <= n; i++)
    // {
    //     for(int j = 1; j <= i; j++)
    //     {
    //         f[i][j] = max(f[i - 1][j], f[i - 1][j - 1]) + a[i][j];
    //     }
    // }

    // 空间优化
    for(int i = 1; i <= n; i++)
    {
```

```
31            for(int j = i; j >= 1; j--) // 修改一下遍历顺序
32            {
33                f[j] = max(f[j], f[j - 1]) + a[i][j];
34            }
35        }
36
37        int ret = 0;
38        for(int j = 1; j <= n; j++)
39        {
40            ret = max(ret, f[j]);
41        }
42
43        cout << ret << endl;
44
45        return 0;
46    }
```

2. 线性 dp

线性dp 是动态规划问题中最基础、最常见的一类问题。它的特点是**状态转移只依赖于前一个或前几个状态**，状态之间的关系是线性的，通常可以用一维或者二维数组来存储状态。

我们在入门阶段解决的《下楼梯》以及《数字三角形》其实都是线性 dp，一个是一维的，另一个是二维的。

2.1 基础线性 dp

2.1.1 台阶问题

> **题目来源：** 洛谷
>
> **题目链接：** P1192 台阶问题
>
> **难度系数：** ★

【题目描述】

> 有 N 级台阶，你一开始在底部，每次可以向上迈 $1 \sim K$ 级台阶，问到达第 N 级台阶有多少种不同方式。

【输入描述】

> 两个正整数 N, K 。

> 对于 100% 的数据， $1 \leq N \leq 100000$，$1 \leq K \leq 100$。

【输出描述】

> 一个正整数 $ans(\mathrm{mod}100003)$，为到达第 N 级台阶的不同方式数。

【示例一】

> 输入：
>
> 5 2
>
> 输出：
>
> 8

【解法】

斐波那契数列模型

1. 状态表示：

   $dp[i]$ 表示：走到 i 位置的方案数。

   那么 $dp[n]$ 就是我们要的结果。

2. 状态转移方程：

   可以从 $i-k \leq j \leq i-1$ 区间内的台阶走到 i 位置，那么总方案数就是所有的 $dp[j]$ 累加在一起。

   注意 $i-k$ 不能小于 0 。

3. 初始化：

   $dp[0]=1$，起始位置，为了让后续填表有意义。

4. 填表顺序：

   从左往右。

【参考代码】

```
#include <iostream>

using namespace std;
```

```cpp

const int N = 1e5 + 10, MOD = 1e5 + 3;

int n, k;
int f[N];

int main()
{
    cin >> n >> k;

    f[0] = 1;
    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= k && i - j >= 0; j++)
        {
            f[i] = (f[i] + f[i - j]) % MOD;
        }
    }

    cout << f[n] << endl;

    return 0;
}
```

2.1.2 最大子段和

> **题目来源：** 洛谷
>
> **题目链接：** P1115 最大子段和
>
> **难度系数：** ★★

【题目描述】

> 给出一个长度为 n 的序列 a ，选出其中连续且非空的一段使得这段和最大。

【输入描述】

> 第一行是一个整数，表示序列的长度 n 。
>
> 第二行有 n 个整数，第 i 个整数表示序列的第 i 个数字 a_i 。
>
> - 对于 40% 的数据，保证 $n \leq 1 \times 10^3$ 。
> - 对于 100% 的数据，保证 $1 \leq n \leq 2 \times 10^5, -10^4 \leq a_i \leq 10^4$ 。

【输出描述】

> 输出一行一个整数表示答案。

【示例一】

> 输入：
>
> 7
>
> 2 -4 3 -1 2 -4 3
>
> 输出：
>
> 4

【解法】

又又又遇见这道题了~

1. 状态表示：

   $dp[i]$ 表示：以 i 位置元素为结尾的「所有子数组」中和的最大值。

   那我们的最终结果应该是 dp 表里面的最大值。

2. 状态转移方程：

   $dp[i]$ 的所有可能可以分为以下两种：

   a. 子数组的长度为 1 ：此时 $dp[i] = a[i]$ ；

   b. 子数组的长度大于 1 ：此时 $dp[i]$ 应该等于以 $i-1$ 为结尾的「所有子数组」中和的最大值再加上 $a[i]$ ，也就是 $dp[i-1] + a[i]$ 。

   应该是两种情况下的最大值，因此可得转移方程： $dp[i] = max(a[i], dp[i-1] + a[i])$ 。

3. 初始化：

   把第一个格子初始化为 0 ，往后填数的时候就不会影响最终结果。

4. 填表顺序：

   根据「状态转移方程」易得，填表顺序为「从左往右」。

【参考代码】

```
#include <iostream>

using namespace std;
```

```cpp

const int N = 2e5 + 10;

int n;
int f[N];

int main()
{
    cin >> n;

    int ret = -1e9;
    for(int i = 1; i <= n; i++)
    {
        int x; cin >> x;
        f[i] = max(f[i - 1] + x, x);
        ret = max(ret, f[i]);
    }

    cout << ret << endl;

    return 0;
}
```

2.1.3 传球游戏

> **题目来源：** 洛谷
>
> **题目链接：** P1057 [NOIP2008 普及组] 传球游戏
>
> **难度系数：** ★★

【题目描述】

> 上体育课的时候，小蛮的老师经常带着同学们一起做游戏。这次，老师带着同学们一起做传球游戏。
>
> 游戏规则是这样的：n 个同学站成一个圆圈，其中的一个同学手里拿着一个球，当老师吹哨子时开始传球，每个同学可以把球传给自己左右的两个同学中的一个（左右任意），当老师再次吹哨子时，传球停止，此时，拿着球没有传出去的那个同学就是败者，要给大家表演一个节目。
>
> 聪明的小蛮提出一个有趣的问题：有多少种不同的传球方法可以使得从小蛮手里开始传的球，传了 m 次以后，又回到小蛮手里。两种传球方法被视作不同的方法，当且仅当这两种方法中，接到球的同学按接球顺序组成的序列是不同的。比如有三个同学 1 号、2 号、3 号，并假设小蛮为 1 号，球传了 3 次回到小蛮手里的方式有 $1\to2\to3\to1$ 和 $1\to3\to2\to1$，共 2 种。

【输入描述】

> 一行，有两个用空格隔开的整数 $n,m(3\le n\le30,1\le m\le30)$。

> 对于 100% 的数据，满足：$3 \le n \le 30, 1 \le m \le 30$。

【输出描述】

> 1 个整数，表示符合题意的方法数。

【示例一】

> 输入：
>
> 3 3
>
> 输出：
>
> 2

【解法】

1. 状态表示：

   $f[i][j]$ 表示传了 i 次，落在第 j 个人手里时的总方案数。

   那么 $f[m][1]$ 就是我们想要的结果。

2. 状态转移方程：

   因为是一个环形结构，第一个位置和最后一个位置可以特殊处理：

   a. 当 $2 \le j \le n-1$ 时，可以从 $j-1$ 或者 $j+1$ 传到该位置，那么总方案数就是 $f[i-1][j-1] + f[i-1][j+1]$；

   b. 当 $j = 1$ 时，可以从 n 或者 2 传到该位置，那么总方案数就是 $f[i-1][n] + f[i-1][2]$；

   c. 当 $j = n$ 时，可以从 $n-1$ 或者 1 传到该位置，那么总方案数就是 $f[i-1][1] + f[i-1][n-1]$。

3. 初始化：

   刚开始的状态设置为 1，让后续填表是正确的，$f[0][1] = 1$。

4. 填表顺序：

   一定要先循环次数，再循环位置。因为我们更新状态是从低次数更新高次数，也就是第一行更新第二行。因此填表顺序应该是从上往下每一行，行的顺序无所谓。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 50;

int n, m;
int f[N][N]; // f[i][j] 表示：传递了 i 次之后，落在了第 j 号同学手里的方案数

int main()
{
    cin >> n >> m;

    f[0][1] = 1;
    for(int i = 1; i <= m; i++)
    {
        // 第一个人
        f[i][1] = f[i - 1][2] + f[i - 1][n];

        // 中间的同学
        for(int j = 2; j < n; j++)
        {
            f[i][j] = f[i - 1][j - 1] + f[i - 1][j + 1];
        }

        // 最后一位同学
        f[i][n] = f[i - 1][1] + f[i - 1][n - 1];
    }

    cout << f[m][1] << endl;

    return 0;
}
```

2.1.4 乌龟棋

> **题目来源：** 洛谷
>
> **题目链接：** P1541 [NOIP2010 提高组] 乌龟棋
>
> **难度系数：** ★★★

【题目描述】

> 小明过生日的时候，爸爸送给他一副乌龟棋当作礼物。

乌龟棋的棋盘是一行 N 个格子，每个格子上一个分数（非负整数）。棋盘第 1 格是唯一的起点，第 N 格是终点，游戏要求玩家控制一个乌龟棋子从起点出发走到终点。

乌龟棋中 M 张爬行卡片，分成 4 种不同的类型（M 张卡片中不一定包含所有 4 种类型的卡片，见样例），每种类型的卡片上分别标有 $1, 2, 3, 4$ 四个数字之一，表示使用这种卡片后，乌龟棋子将向前爬行相应的格子数。游戏中，玩家每次需要从所有的爬行卡片中选择一张之前没有使用过的爬行卡片，控制乌龟棋子前进相应的格子数，每张卡片只能使用一次。

游戏中，乌龟棋子自动获得起点格子的分数，并且在后续的爬行中每到达一个格子，就得到该格子相应的分数。玩家最终游戏得分就是乌龟棋子从起点到终点过程中到过的所有格子的分数总和。

很明显，用不同的爬行卡片使用顺序会使得最终游戏的得分不同，小明想要找到一种卡片使用顺序使得最终游戏得分最多。

现在，告诉你棋盘上每个格子的分数和所有的爬行卡片，你能告诉小明，他最多能得到多少分吗？

【输入描述】

每行中两个数之间用一个空格隔开。

第 1 行 2 个正整数 N, M，分别表示棋盘格子数和爬行卡片数。

第 2 行 N 个非负整数，$a_1, a_2, \ldots, a_N$，其中 a_i 表示棋盘第 i 个格子上的分数。

第 3 行 M 个整数，$b_1, b_2, \ldots, b_M$，表示 M 张爬行卡片上的数字。

输入数据保证到达终点时刚好用光 M 张爬行卡片。

对于 100% 的数据有 $1 \le N \le 350, 1 \le M \le 120$，且 4 种爬行卡片，每种卡片的张数不会超过 40；$0 \le a_i \le 100, 1 \le i \le N, 1 \le b_i \le 4, 1 \le i \le M$。

【输出描述】

一个整数，表示小明最多能得到的分数。

【示例一】

输入：

```
9 5
6 10 14 2 8 8 18 5 17
1 3 1 2 1
```

输出：

```
73
```

【解法】

1. 状态表示：

$f[i][a][b][c][d]$ 表示：走到 i 位置时，编号为 1234 的卡片分别用了 $abcd$ 张，此时的最大分数。

我们发现，当 1234 用的卡片数确定之后，走到的位置 i 可以计算出来，其中 $i = 1 + a + 2b + 3c + 4d$。

因此状态表示可以优化掉一维，变成 $f[a][b][c][d]$，表示：编号为 1234 的卡片分别用了 $abcd$ 张，此时的最大分数。

2. 状态转移方程：

设根据最后一次用的卡片种类，分情况讨论：

a. 如果 $a > 0$，并且最后一张用 1 卡片，最大分数为：$f[a-1][b][c][d] + nums[i]$；

b. 如果 $b > 0$，并且最后一张用 2 卡片，最大分数为：$f[a][b-1][c][d] + nums[i]$；

c. 如果 $c > 0$，并且最后一张用 3 卡片，最大分数为：$f[a][b][c-1][d] + nums[i]$；

d. 如果 $d > 0$，并且最后一张用 4 卡片，最大分数为：$f[a][b][c][d-1] + nums[i]$；

综上所述，取四种情况里面的最大值即可。

3. 初始化：

一张卡片也不用的情况下，可以获得第一个格子的分数，$f[0][0][0][0] = nums[1]$。

4. 填表顺序：

从小到大枚举每种卡片使用的张数即可。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 360, M = 50;

int n, m;
int x[N], cnt[5];
int f[M][M][M][M];

int main()
{
    cin >> n >> m;
```

```
14
15      for(int i = 1; i <= n; i++) cin >> x[i];
16      for(int i = 1; i <= m; i++)
17      {
18          int t; cin >> t;
19          cnt[t]++;
20      }
21
22      // 初始化
23      f[0][0][0][0] = x[1];
24      for(int a = 0; a <= cnt[1]; a++)
25          for(int b = 0; b <= cnt[2]; b++)
26              for(int c = 0; c <= cnt[3]; c++)
27                  for(int d = 0; d <= cnt[4]; d++)
28                  {
29                      int i = 1 + a + 2 * b + 3 * c + 4 * d;
30                      int& t = f[a][b][c][d];
31                      if(a) t = max(t, f[a - 1][b][c][d] + x[i]);
32                      if(b) t = max(t, f[a][b - 1][c][d] + x[i]);
33                      if(c) t = max(t, f[a][b][c - 1][d] + x[i]);
34                      if(d) t = max(t, f[a][b][c][d - 1] + x[i]);
35                  }
36
37      cout << f[cnt[1]][cnt[2]][cnt[3]][cnt[4]] << endl;
38
39      return 0;
40  }
```

2.2 路径类 dp

路径类 dp 是线性 dp 的一种，它是在一个 n × m 的矩阵中设置一个行走规则，研究从起点走到终点的方案数、最小路径和或者最大路径和等等的问题。

入门阶段的《数字三角形》其实就是路径类 dp。

2.2.1 矩阵的最小路径和

> **题目来源：** 牛客网
>
> **题目链接：** DP11 矩阵的最小路径和
>
> **难度系数：** ★

【题目描述】

给定一个 $n \times m$ 的矩阵 a ，从左上角开始每次只能向右或者向下走，最后到达右下角的位置，路径上所有的数字累加起来就是路径和，输出所有的路径中最小的路径和。

数据范围: $1 \leq n, m \leq 500$ ，矩阵中任意值都满足 $0 \leq a_{i,j} \leq 100$

要求：时间复杂度 $O(nm)$

例如：当输入 $[[1,3,5,9],[8,1,3,4],[5,0,6,1],[8,8,4,0]]$ 时，对应的返回值为 12 ，

所选择的最小累加和路径如下图所示：

1	3	5	9
8	1	3	4
5	0	6	1
8	8	4	0

【输入描述】

第一行输入两个正整数 n 和 m 表示矩阵 a 的长宽

后续输入 n 行每行有 m 个数表示矩阵的每个元素

【输出描述】

输出从左上角到右下角的最小路径和

【示例一】

输入：

```
4 4
1 3 5 9
8 1 3 4
5 0 6 1
8 8 4 0
```

8 1 3 4

5 0 6 1

8 8 4 0

输出：

12

【示例二】

输入：

2 3

1 2 3

1 2 3

输出：

7

【解法】

1. 状态表示：

   $dp[i][j]$ 表示：到达 $[i,j]$ 位置处，最小路径和是多少。

   那我们的最终结果就是 $dp[n][m]$ 。

2. 状态转移：

   到达 $[i,j]$ 位置之前的一小步，有两种情况：

   i. 从 $[i-1,j]$ 向下走一步，转移到 $[i,j]$ 位置；

   ii. 从 $[i,j-1]$ 向右走一步，转移到 $[i,j]$ 位置。

   由于到 $[i,j]$ 位置两种情况，并且我们要找的是最小路径，因此只需要这两种情况下的最小值，再加上 $[i,j]$ 位置上本身的值即可：$dp[i][j]=min(dp[i-1][j],dp[i][j-1])+a[i][j]$ 。

3. 初始化：

   第一行和第一列是要初始化的，因为会越界访问。

   但是如果把整张表初始化为无穷大，然后把 $dp[0][1]$ 和 $dp[1][0]$ 的值设为 0 ，后续填表就是正确的。

4. 填表顺序：

根据「状态转移方程」的推导来看，填表的顺序就是「从上往下」填每一行，每一行「从左往后」。

【参考代码】

```cpp
#include <iostream>
#include <cstring>

using namespace std;

const int N = 510;

int n, m;
int f[N][N];

int main()
{
    cin >> n >> m;

    // 初始化
    memset(f, 0x3f, sizeof f);
    f[0][1] = 0;

    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= m; j++)
        {
            int x; cin >> x;
            f[i][j] = min(f[i - 1][j], f[i][j - 1]) + x;
        }
    }

    cout << f[n][m] << endl;

    return 0;
}
```

2.2.2 「木」迷雾森林

> **题目来源：** 牛客网
>
> **题目链接：** 「木」迷雾森林

> **难度系数：** ★

【题目描述】

> 赛时提示：保证出发点和终点都是空地
>
> 帕秋莉掌握了一种木属性魔法
>
> 这种魔法可以生成一片森林（类似于迷阵），但一次实验时，帕秋莉不小心将自己困入了森林
>
> 帕秋莉处于地图的左下角，出口在地图右上角，她只能够向上或者向右行走
>
> 现在给你森林的地图，保证可以到达出口，请问有多少种不同的方案
>
> 答案对 2333 取模

【输入描述】

> 第一行两个整数 m, n 表示森林是 m 行 n 列
> 接下来 m 行，每行 n 个数，描述了地图
> 0 - 空地
> 1 - 树（无法通过）
>
> 对于 100% 的数据， $n, m \leq 3,000$

【输出描述】

> 一个整数表示答案

【示例一】

> 输入：
>
> ```
> 3 3
> 0 1 0
> 0 0 0
> 0 0 0
> ```
>
> 输出：
>
> ```
> 3
> ```

【解法】

1. 状态表示：

   $f[i][j]$ 表示：到达 $[i, j]$ 位置时，有多少种方案。

   那么 $f[1][m]$ 就是我们要的结果。

2. 状态转移方程：

a. 如果 $[i,j]$ 位置是空地，到达 $[i,j]$ 位置有两种方式：

- 从 $[i+1,j]$ 向上走一步，此时的方案数为 $f[i+1][j]$；
- 从 $[i,j-1]$ 向右走一步，此时的方案数为 $f[i][j-1]$。

两者总和就是到达 $[i,j]$ 位置的总方案数。

b. 如果 $[i,j]$ 位置是树，无法走到，$f[i][j]=0$。

3. 初始化：

可以在原始矩阵的规模上多加上一行和一列，把 $f[n+1][1]$ 或者 $f[n][0]$ 初始化为 1，这样后续填表就会有意义。

4. 填表顺序：

从下往上每一行，每一行从左往右。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 3010, MOD = 2333;

int n, m;
int a[N][N];
int f[N][N];

int main()
{
    scanf("%d%d", &n, &m);

    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= m; j++)
            scanf("%d", &a[i][j]);

    // 初始化
    f[n][0] = 1;
    for(int i = n; i >= 1; i--)
        for(int j = 1; j <= m; j++)
            if(a[i][j] == 0)
                f[i][j] = (f[i][j - 1] + f[i + 1][j]) % MOD;
```

```
25
26      cout << f[1][m] << endl;
27
28      return 0;
29  }
```

2.2.3 过河卒

题目来源： 洛谷

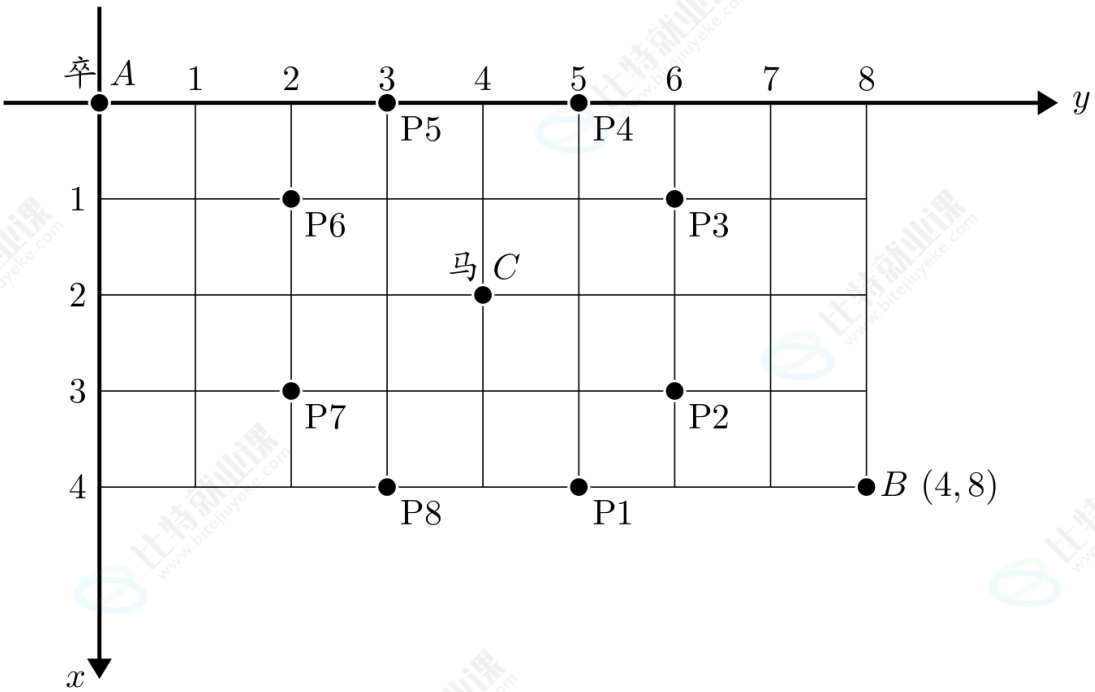
题目链接： P1002 [NOIP2002 普及组] 过河卒

难度系数： ★

【题目描述】

棋盘上 A 点有一个过河卒，需要走到目标 B 点。卒行走的规则：可以向下、或者向右。同时在棋盘上 C 点有一个对方的马，该马所在的点和所有跳跃一步可达的点称为对方马的控制点。因此称之为“马拦过河卒”。

棋盘用坐标表示，A 点 $(0,0)$、B 点 (n,m)，同样马的位置坐标是需要给出的。



现在要求你计算出卒从 A 点能够到达 B 点的路径的条数，假设马的位置是固定不动的，并不是卒走一步马走一步。

【输入描述】

一行四个正整数，分别表示 B 点坐标和马的坐标。

> 对于 100% 的数据， $1 \le n, m \le 20$ ， $0 \le$ 马的坐标 ≤ 20 。

【输出描述】

> 一个整数，表示所有的路径条数。

【示例一】

> 输入：
>
> 6 6 3 3
>
> 输出：
>
> 6

【解法】

1. 状态表示：

   $f[i][j]$ 表示：到达 $[i, j]$ 位置的方案数。

   那么 $f[n][m]$ 就是我们要的结果。

2. 状态转移方程：

   a. 如果 $[i, j]$ 位置能走到，到达 $[i, j]$ 位置之前的一小步，有两种情况：

   - 从 $[i-1, j]$ 向下走一步，走到 $[i, j]$ ，此时的方案数为 $f[i-1][j]$ ；
   - 从 $[i, j-1]$ 向右走一步，走到 $[i, j]$ ，此时的方案数为 $f[i][j-1]$ ；

   那么总方案数 $f[i][j] = f[i-1][j] + f[i][j-1]$ 。

   b. 如果 $[i, j]$ 位置走不到， $f[i][j] = 0$ 。

3. 初始化：

   我们可以给原始的矩阵多加一行多加一列， n, m, x, y 全部 $+1$ ，这样填任何一个位置都不会越界。

   然后初始化 $f[1][0] = 1$ 或者 $f[0][1] = 1$ ，保证后续填表正确即可。

4. 填表顺序：

   从上往下每一行，每一行从左往右。

【参考代码】

```cpp
#include <iostream>

using namespace std;

typedef long long LL;

const int N = 25;

int n, m, a, b;
LL f[N][N];

// 判断是否是被马所拦截的点
bool check(int i, int j)
{
    return (i == a && j == b) || (i != a && j != b && abs(i - a) + abs(j - b) == 3);
}

int main()
{
    cin >> n >> m >> a >> b;
    n++; m++; a++; b++;

    // 初始化
    f[0][1] = 1;
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= m; j++)
        {
            if(check(i, j)) continue;

            f[i][j] = f[i - 1][j] + f[i][j - 1];
        }

    cout << f[n][m] << endl;

    return 0;
}
```

2.2.4 方格取数

> **题目来源：** 洛谷
>
> **题目链接：** P1004 [NOIP2000 提高组] 方格取数
>
> **难度系数：** ★★★

【题目描述】

设有 $N \times N$ 的方格图 $(N \leq 9)$，我们将其中的某些方格中填入正整数，而其他的方格中则放入数字 0 。如下图所示（见样例）：

A 0	0	0	0	0	0	0	0
0	0	13	0	0	6	0	0
0	0	0	0	7	0	0	0
0	0	0	14	0	0	0	0
0	21	0	0	0	4	0	0
0	0	15	0	0	0	0	0
0	14	0	0	0	0	0	0
0	0	0	0	0	0	0	0 B

某人从图的左上角的 A 点出发，可以向下行走，也可以向右走，直到到达右下角的 B 点。在走过的路上，他可以取走方格中的数（取走后的方格中将变为数字 0 ）。
此人从 A 点到 B 点共走两次，试找出 2 条这样的路径，使得取得的数之和为最大。

【输入描述】

输入的第一行为一个整数 N （表示 $N \times N$ 的方格图），接下来的每行有三个整数，前两个表示位置，第三个数为该位置上所放的数。一行单独的 0 表示输入结束。

数据范围：$1 \leq N \leq 9$。

【输出描述】

只需输出一个整数，表示 2 条路径上取得的最大的和。

【示例一】

输入：

```
8
2 3 13
```

```
2 6 6
3 5 7
4 4 14
5 2 21
5 6 4
6 3 15
7 2 14
0 0 0
```

输出：

```
67
```

【解法】

贪心 + 两次 dp 是错误的，因为两次最优不等于全局最优，可以举出反例。正解应该是同时去走两条路，两者相互影响，所以放在一起考虑。

1. 状态表示：

   需要知道当前这两条路径走到什么位置，因此需要四维 $f[i_1][j_1][i_2][j_2]$ 来表示第一条路走到 $[i_1, j_1]$ 第二条路走到 $[i_2][j_2]$。

   但是我们发现，因为两者是同时出发的，所以横纵坐标之和是一个定值。也就是说，只要知道了横纵坐标之和，以及两者的横坐标，就可以计算出纵坐标，状态表示就可以优化掉一维。

   优化后的状态表示：$f[st][i_1][i_2]$ 表示：第一条路在 $[i_1, st - i_1]$，第二条路在 $[i_2, st - i_2]$ 时，两者的路径最大和。那我们的最终结果就是 $f[n \times 2][n][n]$。

2. 状态转移方程：

   第一条路可以从上 $[i_1 - 1, st - i_1]$ 或者左 $[i_1, st - i_1 - 1]$ 走到 $[i_1, st - i_1]$ 位置；第二条路可以从上 $[i_2 - 1, st - i_2]$ 或者左 $[i_2, st - i_2 - 1]$ 走到 $[i_2, st - i_2]$ 位置。排列组合一下一共 4 中情况，分别是：

   - 上 + 上，此时的最大和为：$f[st - 1][i_1 - 1][i_2 - 1]$；
   - 上 + 左，此时的最大和为：$f[st - 1][i_1 - 1][i_2]$；
   - 左 + 上，此时的最大和为：$f[st - 1][i_1][i_2 - 1]$；
   - 左 + 左，此时的最大和为：$f[st - 1][i_1][i_2]$；

   取上面四种情况的最大值，然后再加上 $a[i_1][j_1]$ 和 $a[i_2][j_2]$。但是要注意，如果两个路径当前在同一位置时，只用加上一个 $a[i_1][j_1]$ 即可。

3. 初始化：

   算的是路径和，0 不会影响最终结果，直接填表。

4. 填表顺序：

   先从小到大循环横纵坐标之和，然后依次从小到大循环两者的横坐标。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 15;

int n;
int a[N][N];
int f[N * 2][N][N];

int main()
{
    cin >> n;
    int x, y, w;
    while(cin >> x >> y >> w, x)
    {
        a[x][y] = w;
    }

    for(int s = 2; s <= n + n; s++)
    {
        for(int i1 = 1; i1 <= n; i1++)
        {
            for(int i2 = 1; i2 <= n; i2++)
            {
                int j1 = s - i1, j2 = s - i2;
                if(j1 <= 0 || j1 > n || j2 <= 0 || j2 > n) continue;

                int t = f[s - 1][i1][i2];
                t = max(t, f[s - 1][i1][i2 - 1]);
                t = max(t, f[s - 1][i1 - 1][i2]);
                t = max(t, f[s - 1][i1 - 1][i2 - 1]);

```

```
34                  if(i1 == i2)
35                  {
36                      f[s][i1][i2] = t + a[i1][j1];
37                  }
38                  else
39                  {
40                      f[s][i1][i2] = t + a[i1][j1] + a[i2][j2];
41                  }
42              }
43          }
44      }
45
46      cout << f[n + n][n][n] << endl;
47
48      return 0;
49  }
```

2.3 经典线性 dp

经典线性 dp 问题有两个：最长上升子序列（简称：LIS）以及最长公共子序列（简称：LCS），这两道题目的很多方面都是可以作为经验，运用到别的题目中。比如：解题思路，定义状态表示的方式，推到状态转移方程的技巧等等。

因此，这两道经典问题是一定需要掌握的。

2.3.1 最长上升子序列（一）

> **题目来源：** 洛谷
>
> **题目链接：** B3637 最长上升子序列
>
> **难度系数：** ★

【题目描述】

> 这是一个简单的动规板子题。
>
> 给出一个由 $n(n \leq 5000)$ 个不超过 10^6 的正整数组成的序列。请输出这个序列的最长上升子序列的长度。
>
> 最长上升子序列是指，从原序列中按顺序取出一些数字排在一起，这些数字是逐渐增大的。

【输入描述】

> 第一行，一个整数 n，表示序列长度。
>
> 第二行有 n 个整数，表示这个序列。

【输出描述】

> 一个整数表示答案。

【示例一】

> 输入：
>
> 6
>
> 1 2 4 1 3 4
>
> 输出：
>
> 4

【解法】

1. 状态表示

   $dp[i]$ 表示：以 i 位置元素为结尾的「所有子序列」中，最长递增子序列的长度。

   最终结果就是整张 dp 表里面的最大值。

2. 状态转移方程：

   对于 $dp[i]$，我们可以根据「子序列的构成方式」，进行分类讨论：

   - 子序列长度为 1：只能自己玩了，此时 $dp[i] = 1$；
   - 子序列长度大于 1：$a[i]$ 可以跟在前面某些数后面形成子序列。设前面的某一个数的下标为 j，其中 $1 \le j < i - 1$。只要 $a[j] < a[i]$，i 位置元素跟在 j 元素后面就可以形成递增序列，长度为 $dp[j] + 1$。

     因此，我们仅需找到满足要求的最大的 $dp[j] + 1$ 即可。

   综上，$dp[i] = max(dp[j] + 1, dp[i])$，其中 $1 \le j < i \ \&\& \ nums[j] < nums[i]$。

3. 初始化：

   不用单独初始化，每次填表的时候，先把这个位置的数改成 1 即可。

4. 填表顺序：

   显而易见，填表顺序「从左往右」。

【参考代码】

```
1  #include <iostream>
2
```

```cpp
using namespace std;

const int N = 5010;

int n;
int a[N];
int f[N];

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];

    int ret = 0;
    for(int i = 1; i <= n; i++)
    {
        f[i] = 1; // 长度为 1 的子序列
        for(int j = 1; j < i; j++)
        {
            if(a[j] < a[i])
            {
                f[i] = max(f[i], f[j] + 1);
            }
        }
        ret = max(ret, f[i]);
    }

    cout << ret << endl;

    return 0;
}
```

2.3.2 最长上升子序列（二）

> **题目来源：** 牛客网
>
> **题目链接：** 【模板】最长上升子序列
>
> **难度系数：** ★★

【题目描述】

> 给你一个长度为 n 的数组，求最长的严格上升子序列的长度。

【输入描述】

> 第一行一个整数 n ，表示数组长度。

> 第二行 n 个整数，表示数组中的元素。
>
> $1 \leq n \leq 100000$

【输出描述】

> 输出一行，表示答案。

【示例一】

> 输入：
>
> 5
>
> 1 2 2 2 3
>
> 输出：
>
> 3

【解法】

利用贪心 + 二分优化动态规划：

- 我们在考虑最长递增子序列的长度的时候，其实并不关心这个序列长什么样子，我们只是关心最后一个元素是谁。这样新来一个元素之后，我们就可以判断是否可以拼接到它的后面。
- 因此，我们可以创建一个数组，统计长度为 `x` 的递增子序列中，最后一个元素是谁。为了尽可能的让这个序列更长，我们仅需统计长度为 `x` 的所有递增序列中最后一个元素的「最小值」。
- 统计的过程中发现，数组中的数呈现「递增」趋势，因此可以使用「二分」来查找插入位置。

【参考代码】

```
#include <iostream>

using namespace std;

const int N = 1e5 + 10;

int n;
int a[N];
int f[N], len; // 注意与上一个动态规划数组的含义是不一样的

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];

```

```
    for(int i = 1; i <= n; i++)
    {
        // 处理边界情况
        if(len == 0 || a[i] > f[len]) f[++len] = a[i];
        else
        {
            // 二分插入位置
            int l = 1, r = len;
            while(l < r)
            {
                int mid = (l + r) / 2;
                if(f[mid] >= a[i]) r = mid;
                else l = mid + 1;
            }
            f[l] = a[i];
        }
    }

    cout << len << endl;

    return 0;
}
```

2.3.3 合唱队形

题目来源： 洛谷

题目链接： P1091 [NOIP2004 提高组] 合唱队形

难度系数： ★★

【题目描述】

n 位同学站成一排，音乐老师邀请其中的 $n-k$ 位同学出列，使得剩下的 k 位同学排成合唱队形。

合唱队形是指这样的一种队形：设 k 位同学从左到右依次编号为 $1,2,\ldots,k$，他们的身高分别为 $t_1,t_2,\ldots,t_k$，则他们的身高满足 $t_1<\cdots<t_i>t_{i+1}>ti+1>\ldots>t_k(1\le i\le k)$。

你的任务是，已知所有 n 位同学的身高，计算最少需要几位同学出列，可以使得剩下的同学排成合唱队形。

【输入描述】

共二行。

第一行是一个整数 n（$2\le n\le 100$），表示同学的总数。

> 第二行有 n 个整数，用空格分隔，第 i 个整数 $t_i(130 \leq t_i \leq 230)$ 是第 i 位同学的身高（厘米）。
>
> 对于全部的数据，保证有 $n \leq 100$。

【输出描述】

> 一个整数，最少需要几位同学出列。

【示例一】

> 输入：
>
> 8
>
> 186 186 150 200 160 130 197 220
>
> 输出：
>
> 4

【解法】

对于每一个位置 i ，计算：

- 从左往右看：以 i 为结尾的最长上升子序列 $f[i]$；
- 从右往左看：以 i 为结尾的最长上升子序列 $g[i]$；

最终结果就是所有 $f[i]+g[i]-1$ 里面的最大值。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 110;

int n;
int a[N];
int f[N], g[N];

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];

    // 从左往右
    for(int i = 1; i <= n; i++)
```

```
    {
        f[i] = 1;
        for(int j = 1; j < i; j++)
        {
            if(a[j] < a[i])
            {
                f[i] = max(f[i], f[j] + 1);
            }
        }
    }

    // 从右往左
    for(int i = n; i >= 1; i--)
    {
        g[i] = 1;
        for(int j = n; j > i; j--)
        {
            if(a[j] < a[i])
            {
                g[i] = max(g[i], g[j] + 1);
            }
        }
    }

    int ret = 0;
    for(int i = 1; i <= n; i++)
    {
        ret = max(ret, f[i] + g[i] - 1);
    }

    cout << n - ret << endl;

    return 0;
}
```

2.3.4 牛可乐和最长公共子序列

> **题目来源：** 牛客网
>
> **题目链接：** 牛可乐和最长公共子序列
>
> **难度系数：** ★★

【题目描述】

> 牛可乐得到了两个字符串 s 和 t ,牛可乐想请聪明的你帮他计算出来，两个字符串的最长公共子序列长度是多少。
>
> 最长公共子序列的定义是，子序列中的每个字符都能在两个原串中找到，而且每个字符的先后顺序和原串中的先后顺序一致。

【输入描述】

> 输入包含多组数据，请读至文件末尾。
>
> 每行包含两个字符串 s,t ，两个字符串用一个空格字符间隔，单个字符串长度不超过 5000 。
>
> 数据保证所有数据的字符串 s 长度之和与字符串 t 长度之和均不超过 5000 。

【输出描述】

> 对于每组数据，输出一个整数，代表最长公共子序列的长度。

【示例一】

> 输入：
>
> abccde bcee
>
> 输出：
>
> 3

【解法】

1. 状态表示：

   $dp[i][j]$ 表示： $s1$ 的 $[1,i]$ 区间以及 $s2$ 的 $[1,j]$ 区间内的所有的子序列中，最长公共子序列的长度。

   那么 $dp[n][m]$ 就是我们要的结果。

2. 状态转移方程：

   对于 $dp[i][j]$ ，我们可以根据 $s1[i]$ 与 $s2[j]$ 的字符分情况讨论：

   a. 两个字符相同 $s1[i]=s2[j]$ ：那么最长公共子序列就在 $s1$ 的 $[1,i-1]$ 以及 $s2$ 的 $[1,j-1]$ 区间上找到一个最长的，然后再加上 $s1[i]$ 即可。因此 $dp[i][j]=dp[i-1][j-1]+1$;

   b. 两个字符不同 $s1[i]\neq s2[j]$ ：那么最长公共子序列一定不会同时以 $s1[i]$ 和 $s2[j]$ 结尾。那么我们找最长公共子序列时，有下面三种策略：

   - 去 $s1$ 的 $[1,i-1]$ 以及 $s2$ 的 $[1,j]$ 区间内找：此时最大长度为 $dp[i-1][j]$;
   - 去 $s1$ 的 $[1,i]$ 以及 $s2$ 的 $[1,j-1]$ 区间内找：此时最大长度为 $dp[i][j-1]$;
   - 去 $s1$ 的 $[1,i-1]$ 以及 $s2$ 的 $[1,j-1]$ 区间内找：此时最大长度为 $dp[i-1][j-1]$ 。

我们要三者的最大值即可。但是我们仔细观察会发现，第三种包含在第一种和第二种情况里面，但是我们求的是最大值，并不影响最终结果。因此只需求前两种情况下的最大值即可。

综上，状态转移方程为：

$if(s1[i] = s2[j])\ dp[i][j] = dp[i-1][j-1] + 1$;

$if(s1[i] \neq s2[j])\ dp[i][j] = max(dp[i-1][j], dp[i][j-1])$。

3. 初始化：

   直接填表即可。

4. 填表顺序：

   根据「状态转移方程」得：从上往下填写每一行，每一行从左往右。

【参考代码】

```
#include <iostream>

using namespace std;

const int N = 5010;

string s, t;
int f[N][N];

int main()
{
    while(cin >> s >> t)
    {
        int n = s.size(), m = t.size();
        // s = " " + s, t = " " + t;

        for(int i = 1; i <= n; i++)
        {
            for(int j = 1; j <= m; j++)
            {
                if(s[i - 1] == t[j - 1]) f[i][j] = f[i - 1][j - 1] + 1;
                else f[i][j] = max(f[i - 1][j], f[i][j - 1]);
            }
        }

        cout << f[n][m] << endl;
```

```
27        }
28
29        return 0;
30    }
```

2.3.5 编辑距离

> **题目来源：** 洛谷
>
> **题目链接：** P2758 编辑距离
>
> **难度系数：** ★★★

【题目描述】

> 设 A 和 B 是两个字符串。我们要用最少的字符操作次数，将字符串 A 转换为字符串 B。这里所说的字符操作共有三种：
>
> 1. 删除一个字符；
> 2. 插入一个字符；
> 3. 将一个字符改为另一个字符。
>
> A, B 均只包含小写字母。

【输入描述】

> 第一行为字符串 A；第二行为字符串 B；字符串 A, B 的长度均小于 2000。

【输出描述】

> 只有一个正整数，为最少字符操作次数。

【示例一】

> 输入：
>
> sfdqxbw
>
> gfdgw
>
> 输出：
>
> 4

【解法】

两个字符串之间的 dp 问题，与最长公共子序列的分析方式类似。

1. 状态表示：

   $dp[i][j]$ 表示：字符串 A 中 $[1, i]$ 区间与字符串 B 中 $[1, j]$ 区间内的编辑距离。

   那么 $dp[n][m]$ 就是我们要的结果。

2. 状态转移方程：

   对于 $dp[i][j]$ ，我们可以根据 $A[i]$ 与 $B[j]$ 的字符分情况讨论：

   a. 两个字符相同 $A[i] = B[j]$ ：那么 $dp[i][j]$ 就是 A 的 $[1, i-1]$ 以及 B 的 $[1, j-1]$ 区间内编辑距离，因此 $dp[i][j] = dp[i-1][j-1]$ ；

   b. 两个字符不同 $A[i] \neq B[j]$ ：那么对于 A 字符串，我们可以进行下面三种操作：

   - 删掉 $A[i]$ ：此时 $dp[i][j]$ 就是 A 的 $[1, i-1]$ 以及 B 的 $[1, j]$ 区间内的编辑距离，因此 $dp[i][j] = dp[i-1][j] + 1$ ；
   - 插入一个字符：在字符串 A 的后面插入一个 $B[j]$ ，此时的 $dp[i][j]$ 就是 A 的 $[1, i]$ 以及 B 的 $[1, j-1]$ 区间内的编辑距离，因此 $dp[i][j] = dp[i][j-1] + 1$ ；
   - 将 $A[i]$ 替换成 $B[j]$ ：此时的 $dp[i][j]$ 就是 A 的 $[1, i-1]$ 以及 B 的 $[1, j-1]$ 区间内的编辑距离，因此 $dp[i][j] = dp[i-1][j-1] + 1$ 。

   我们要三者的最小值即可。

3. 初始化：

   需要注意，当 i, j 等于 0 的时候，这些状态也是有意义的。我们可以全部删除，或者全部插入让两者相同。

   因此需要初始化第一行 $dp[0][j] = j\ (1 \leq j \leq m)$ ，第一列 $dp[i][0] = i\ (1 \leq i \leq n)$ 。

4. 填表顺序：

   初始化完之后，从 $[1, 1]$ 位置开始从上往下每一行，每一行从左往右填表即可。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 2010;

string a, b;
int n, m;
int f[N][N];

int main()
{
```

```
13      cin >> a >> b;
14      n = a.size(); m = b.size();
15      a = " " + a; b = " " + b;
16
17      // 初始化
18      for(int i = 1; i <= n; i++) f[i][0] = i;
19      for(int j = 1; j <= m; j++) f[0][j] = j;
20
21      for(int i = 1; i <= n; i++)
22          for(int j = 1; j <= m; j++)
23          {
24              if(a[i] == b[j]) f[i][j] = f[i - 1][j - 1];
25              else f[i][j] = min(min(f[i - 1][j], f[i - 1][j - 1]), f[i][j -
    1]) + 1;
26          }
27
28      cout << f[n][m] << endl;
29
30      return 0;
31  }
```

3. 背包问题

背包问题是动态规划中最经典的问题，很多题目或多或少都有背包问题的影子。它的基本形式是：给定一组物品，每个物品有体积和价值，在不超过背包容量的情况下，选择物品使得总价值最大。

背包问题有多种变体，主要包括：

1. **01 背包问题**：每种物品只能选或不选（选 0 次或 1 次）。
2. **完全背包问题**：每种物品可以选择无限次。
3. **多重背包问题**：每种物品有数量限制。
4. **分组背包问题**：物品被分为若干组，每组只能选一个物品。
5. **混合背包**：以上四种背包问题混在一起。
6. **多维费用的背包问题**：限定条件不止有体积，还会有其他因素（比如重量）。

除了经典的总价值最大问题，还会有：

1. **方案总数。**
2. **最优方案。**
3. **方案可行性。**